Ders 14 Genişletilmiş Notu: Quiz Hataları, Yığınla Kuyruk ve First Child / Next Sibling Ağaç Gösterimi

Ders 14 Genişletilmiş Notu: Quiz Hataları, Yığınla Kuyruk ve First Child / Next Sibling Ağaç Gösterimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Oturumun Akışı
- 2. Yığınla Şifre Üretme: Kaç Kez Pop Yapılır?
- 3. Doğru/Yanlış Soruları
- 4. Sıralı Diziye Eleman Ekleme Sorusu
- 5. Labirentte Çıkış Arayan Fare
- 6. Yığın ve Ters Çevirmeyle Kuyruk
- 7. Otomatik Bilinmesi Gereken Sorular
- 8. Ödev: Ağaçla Sanal Dizin Sistemi
- 9. First Child / Next Sibling Gösterimi
- 10. Komutlar Nasıl Alınır, Ne Yapar?
- 11. Hata Durumları
- 12. Bir Üst Dizine Çıkmak
- 13. Sınırlar: Tek Seviye Hareket ve Bellek
- 14. Neden Aslında Bir Bağlı Liste Ödevi?
- 15. Algoritmik Ders Nasıl Çalışılır?

1. Oturumun Akışı

Bu oturumda yeni bir konuya geçmeden önce ikinci quizde en çok hata yapılan sorulara bakıldı. Ardından verilecek ödev anlatıldı. Ödev, ikili olmayan bir ağacın işaretçilerle nasıl gösterileceğini öğretiyor. Bu yüzden dersin asıl yeni kavramı ilk çocuk / sonraki kardeş (first child / next sibling) gösterimidir.

2. Yığınla Şifre Üretme: Kaç Kez Pop Yapılır?

Quizde en çok hata yapılan sorulardan biri “iki casus” problemiydi. Bu soruda yığına (stack) itme (push) ve yığından çekme (pop) işlemleriyle şifreli bir metin elde ediliyordu. Hata, kaç kez pop yapılacağı sorusundaydı.

Hoca soruyu ekrana yazdı. Girdi şuydu:

```
a b c d e 2 f g h 1 i j k 0 l m n 3 o p r s 1
```

Kural basittir: harfler yığına itilir, bir rakam gelince o sayı kadar eleman üstten çekilip metne eklenir. Hoca ilk adımları gösterdi. `a b c d e` yığına yerleştirilir. `2` gelince üstteki iki eleman çıkar. `f g h` yerleştirilir. `1` gelince `h` çıkar. Aşağıdaki tablo bu izi girdinin sonuna kadar sürdürür (ara adım eklendi).

Girdi parçası	Rakamla yapılan pop	Toplam pop	Yığın (alttan üste)	Oluşan metin
`a b c d e, 2`	`e, d`	2	`a b c`	`ed`
`f g h, 1`	`h`	3	`a b c f g`	`edh`
`i j k, 0`	yok	3	`a b c f g i j k`	`edh`
`l m n, 3`	`n, m, l`	6	`a b c f g i j k`	`edhnml`
`o p r s, 1`	`s`	7	`a b c f g i j k o p r`	`edhnmls`

Öğrencilerin çoğu yalnızca rakamlarla yapılan çekmeleri topladı. Hocanın saydığı gibi 2, 3, 6, 7 diye ilerleyip 7 cevabını yazdılar:

$$2 + 1 + 0 + 3 + 1 = 7$$

Oysa girdi bittiğinde yığında hâlâ eleman vardır. Hoca bunu "diyelim ki yığında en son `a b c` kaldıysa" diye örnekledi. Metni tamamlamak için kalan elemanlar da metne eklenmelidir. Bunun tek yolu onları da pop etmektir. Bu girdide kalan eleman sayısı 11'dir, yani (ara adım eklendi):

$$7 + 11 = 18$$

Girdide 18 harf vardır, yani 18 kez push yapılır. Bastığımız her eleman sonunda çıkacağı için pop sayısı da 18'dir. Kalanlar da çekildiğinde oluşan şifreli metin şudur (ara adım eklendi):

`edhnmlsrpokjigfcba`

Hoca bunun yanıltmaca olsun diye sorulmadığını söyledi. Kalanların da çekilmesinin doğal olduğunu düşünüyorlardı, ama bunu çok az öğrenci fark etti.

> **Hocanın vurgusu:** Bastığınız kadar elemanı en sonunda pop etmeniz gerekir. Yığında ne kaldıysa, kodu oluşturmak için onları da pop etmek zorundasınız.

3. Doğru/Yanlış Soruları

Bu soruda da bütün şıkları doğru yapan öğrenci sayısı çok azdı. Hoca şıkları tek tek ve öğrencilerle birlikte cevapladı.

3.1 Derleyiciler

"Derleyiciler bazı işlemler için ağaç yapısı kullanabilirler" ifadesi doğrudur. "Derleyiciler bazı işlemler için yığın yapısı kullanabilirler" ifadesi de doğrudur. Hoca bu iki ifadenin gerekçesini derste açmadı.

> **Ek bilgi (derste anlatılmadı):** Soruda kastedilen büyük olasılıkla, derleyicinin kaynak kodu çözümlerken kurduğu sözdizimi ağacı (syntax tree) ile fonksiyon çağrıları ve iç içe yapıların izlenmesinde kullanılan yığındır.

3.2 Dosya Sistemleri

"Dosya sistemleri ikili ağaç yapısından faydalanır" ifadesi yanlıştır. Dosya sistemleri ağaç yapısı kullanır, fakat bu ağaç ikili ağaç (binary tree) değildir. Bu ayrım dersin ikinci yarısındaki ödevin de temelidir: bir dizinin altında ikiden fazla dizin olabilir.

3.3 Çift Yönlü Bağlı Listede Silme

"Çift yönlü bağlı listede (double linked list) yer alan herhangi bir elemanı silmek için silinecek elemanın adres bilgisi yeterlidir" ifadesi doğrudur.

Hoca bunu çift yönlü listenin avantajıyla açıkladı. Bir düğüm, kendisinden önceki düğümün adresini de tutar. Düğümde veri, önceki adres ve sonraki adres vardır. Diyelim ki silinecek eleman 1000 numaralı adrestedir. Bu adrese geldiğinizde önceki düğümü de görürsünüz. Bu yüzden önceki düğümün adresini ayrıca tutmaya gerek yoktur ve silme işlemi gerçekleştirilebilir.

> **Ek bilgi (derste anlatılmadı):** Silinecek düğüm `p` ise iki bağ güncellenir: `p->prev->next = p->next` ve `p->next->prev = p->prev`. Düğüm listenin başı ya da sonuysa ilgili komşu `NULL` olacağından bu atamalar kontrol edilerek yapılır. Baş düğüm silindiğinde listenin baş işaretçisi de güncellenir.

4. Sıralı Diziye Eleman Ekleme Sorusu

Bir başka şık şöyleydi: n elemanlı bir dizinin $n+1$. gözüne yeni bir eleman eklenir. Bu durumdaki dizi, araya sokmalı sıralama (insertion sort) ve kabarcık sıralama (bubble sort) algoritmalarıyla en kötü durumda $O(n)$ karmaşıklıkla sıralanır. Doğru mu, yanlış mı?

Hoca soruyu tahtada açtı. İlk n eleman sıralıdır, yeni eleman ise $n+1$. göze yerleştirilir. Böylece dizinin tamamı artık sıralı olmayabilir. Hoca yeni eleman olarak 8'i aldı.

Hoca öğrencilere insertion sort'ta ne olacağını sordu, cevap gelmeyince soruyu öğrencilere bıraktı. Bu nedenle cevap derste verilmedi.

> **Ders sonrası not:** Soru derste cevaplanmadı; aşağıdaki çözüm sonradan eklendi ve sayımlar programla doğrulandı. Insertion sort için ifade doğrudur. Yeni eleman en küçükse sıralı kısımdaki $n-1$ eleman birer karşılaştırma, yeni eleman n karşılaştırma yapar. Toplam $2n-1$ karşılaştırma ve n kaydırma, yani $O(n)$. Her geçişte büyük elemanı sona taşıyan bubble sort'ta ise yeni eleman her geçişte yalnız bir adım sola gider. Yeni eleman en küçükse n geçişin hepsinde yer değiştirme olur, erken çıkış çalışmaz ve karşılaştırma sayısı $n(n+1)/2$, yani $O(n^2)$ olur. Bu sürüm için ifadenin tamamı yanlıştır. Küçük elemanı başa taşıyan, sağdan sola geçişli ve yer değiştirme bayraklı bir sürüm ise iki geçişte, $2n-1$ karşılaştırmayla biter. Bu yüzden cevap, bubble sort'un hangi sürümünün kastedildiğine bağlıdır.

Hocanın örneği olan `3 5 7 9 11 8` dizisinde iz şöyledir (ara adım eklendi):

Algoritma	İz	Karşılaştırma
Insertion sort	Sıralı kısım için 4 karşılaştırma. `8`, `11` ve `9` ile karşılaştırılıp kayar, `7` ile karşılaştırılıp durur.	7
Bubble sort, soldan sağa, bayraklı	1. geçiş: `3 5 7 9 8 11`. 2. geçiş: `3 5 7 8 9 11`. 3. geçişte yer değiştirme olmaz, biter.	5 + 4 + 3 = 12

5. Labirentte Çıkış Arayan Fare

"Labirentte çıkışı arayan fare probleminin çözümü için en uygun veri yapısı" sorusunda da çok eksik vardı. Hoca, seçeneklerin aslında o güne kadar görülen yapılar olduğunu hatırlattı: bağlı liste, yığın, kuyruk (queue) ve belki ağaç.

Beklenen cevap yığındı. Çünkü yol tıkalıysa fare geri gidecektir. Bulunduğu noktadan iki üç adım geriye dönecek ve oradan yola devam etmeye çalışacaktır. Bu tıpkı yığın mantığıdır.

Bazı öğrenciler graf (graph) cevabını verdi. Graf bu derste henüz görülmediği için beklenen cevap değildi, ama labirent graf olarak gösterilebileceği için hoca bu cevabı da doğru saydı.

Hoca ayrıca `rear + 1` ile ilgili soruda da çok hata olduğunu, bunu önceki derste anlattığını belirtti ve burada tekrar açmadı.

> **Hocanın vurgusu:** Soruları dikkatle okumaya çalışın.

6. Yığın ve Ters Çevirmeyle Kuyruk

Soru şuydu: push, pop ve elemanların sırasını ters çevirme (reverse) işlemleri mümkünse kuyruk yapısı nasıl gerçeklenir?

Tahtadaki yığında alttan üste `3 5 7 19` vardır ve kuyruğun başı `3`'tür.

Kuyruğa eleman ekleme (`enQueue`) için doğrudan push yapmak yeterlidir, çünkü yığının üstü kuyruğun sonudur. Kuyruktan eleman çekme (`deQueue`) için üç adım gerekir:

1. Yığın ters çevrilir. Alttan üste `19 7 5 3` olur.
2. Üstteki eleman pop edilir. Çıkan eleman `3`'tür, yani kuyruğun başı. Yığında `19 7 5` kalır.
3. Yığın tekrar ters çevrilir. Alttan üste `5 7 19` olur (ara adım eklendi). Böylece sonraki `enQueue` yine kuyruğun sonuna eklenir.

Hocanın özetiyle: kuyruğa eleman yerleştirirken yığın olduğu gibi kullanılıp yalnız push edilir. Kuyruktan eleman çekerken önce ters çevrilir, eleman çekilir, sonra tekrar ters çevrilir.

> **Ek bilgi (derste anlatılmadı):** Ters çevirme n elemanın hepsine dokunuyorsa bu yöntemde `enQueue` sabit süre, `deQueue` ise iki ters çevirme nedeniyle $O(n)$ sürer.

7. Otomatik Bilinmesi Gereken Sorular

Soru incelemesinin sonunda hoca quizdeki soruların iki türe ayrıldığını anlattı. Bir tür, artık otomatik cevaplanması beklenen sorulardır. Örnekleri şunlardı: sırasız bir dizide eleman aramanın karmaşıklığının n, ikili aramanın (binary search) karmaşıklığının $\log n$ olması ve bir ağacı önce kök sırasıyla (preorder) gezmek. Diğer tür ise düşünmeyi gerektiren sorulardır, yığınla kuyruk sorusu gibi.

8. Ödev: Ağaçla Sanal Dizin Sistemi

Ödevde bir dizinde `mkdir`, `chdir`, `rmdir` ve `count` komutlarını ağaç veri yapısıyla gerçekleştiren bir yapı tasarlanması isteniyor. Hoca sonradan bunlara `dir` komutunu da ekledi. Bu, işletim sisteminin dosya sisteminin (file system) çok küçük bir parçasının taklididir.

> **Ders sonrası not:** Hoca "count dahil dört komut var" dedi; altyazıda sayılan ve görevi anlatılan komutlar `mkdir`, `chdir`, `rmdir`, `dir` ve `count` olmak üzere beştir.

8.1 Örnek Akış

Program kökte (root) başlar. Hocanın tahtada izlediği akış şöyledir. Son sütun, komuttan sonra kökün çocuklarını gösterir.

Komut	Sonuç	Kökün çocukları
`mkdir A`	Kökün altında A açılır.	A
`mkdir B`	Kökün altında B açılır.	A, B
`chdir A`	Artık A'dasınız.	A, B
`mkdir A1`, `mkdir A2`, `mkdir A3`	Üçü de A'nın altında açılır.	A, B
`chdir ..`	Köke dönülür.	A, B
`mkdir C`	Kökün altında C açılır.	A, B, C
`rmdir B`	B silinir.	A, C
`count`	Bulunulan dizinin bir altındaki dizin sayısı: 2.	A, C
`dir`	Yalnız bir alt seviye listelenir, A'nın altı gösterilmez.	A, C

> **Ders sonrası not:** Hoca bu noktada `dir` çıktısı için "A ve B diyecek" dedi; B silindiği için doğrusu A ve C'dir. Hemen önceki `count` için kendisi de "A ve C var, iki diyecek" demişti. Akış, programla çalıştırılarak doğrulandı.

Hoca sonraki anlatımlarında B'yi silinmemiş kabul etti ve kökün altında A, B ve C olan ağacı kullandı. Bu durumda kökteyken `count` 3 verir. A'dayken `dir` yazılırsa ekrana A1, A2 ve A3 yazılır.

9. First Child / Next Sibling Gösterimi

9.1 Sorun ve Fikir

İkili ağaca alışkındık. Orada her düğümün en fazla iki çocuğu olur. Burada ise kökün altında ikiden fazla alt ağaç, A'nın altında da ikiden fazla düğüm vardır. Böyle bir ağaç bağlı liste (linked list) yapısıyla gösterilebilir. Kullanılan yapının adı first child / next sibling gösterimidir.

Fikir şudur: her düğümde iki işaretçi vardır. Biri düğümün **ilk çocuğunu**, diğeri kendisinden sonra gelen **kardeşini** gösterir. Kök; A, B ve C'nin üçünü birden çocuk olarak göstermez. Yalnız ilk eklenen A'yı gösterir. Diğer çocuklara A üzerinden ulaşılır: A, B'yi kardeş olarak gösterir, B de C'yi gösterir.

9.2 Düğüm Yapısı

Her düğümde dizinin adı, ilk çocuk işaretçisi ve sonraki kardeş işaretçisi tutulur. Hoca ad için ödevdeki `data struct` adlı dizini örnek verdi.

```
struct Node
{
    char name[40];
    struct Node *firstChild;
    struct Node *nextSibling;
};
```

9.3 Örnek Ağacın İşaretçileri

Kökün altında A, B, C ve A'nın altında A1, A2, A3 olan ağaç şöyle gösterilir:

```
Ağaç:                    Bağlı gösterim:

      root               root (sibling = NULL)
     /  |  \               |
    A   B   C              child
   /|\                     v
 A1 A2 A3                  A --sibling--> B --sibling--> C --> NULL
                           |
                           child
                           v
                           A1 --sibling--> A2 --sibling--> A3 --> NULL
```

Her düğümün iki işaretçisi tek tek şöyledir:

Düğüm	`firstChild`	`nextSibling`
root	A	NULL
A	A1	B
B	NULL	C
C	NULL	NULL
A1	NULL	A2
A2	NULL	A3
A3	NULL	NULL

Üç noktaya dikkat etmek gerekir:

- Kökün kardeşi yoktur. Kök tepedeki tek düğümdür, bu yüzden `nextSibling` alanı her zaman `NULL` olur.
- Kardeşler oluşturulma sırasıyla uç uca eklenir. Önce A, sonra B, sonra C oluşturulduğu için A'nın kardeşi B, B'nin kardeşi C olur. Yeni gelen C'nin sağına eklenir.
- A3'ün altında `A3_1` açılsaydı A3'ün `firstChild` alanı onu gösterirdi. Mantık bütün seviyelerde aynıdır.

10. Komutlar Nasıl Alınır, Ne Yapar?

Soru: Bu komutlar yalnızca kod içinde çağrılan fonksiyonlar mı, yoksa konsoldan mı yazılacak?

Cevap: Girdi olarak dışarıdan yazılacak. Dışarıdan `mkdir A` geldiğinde A adlı dizini oluşturma işi yapılır. `chdir A` geldiğinde A'ya geçilir, `rmdir B` geldiğinde B silinir. Bu işler tek bir yığın kod olarak değil, her komut için ayrı fonksiyon olarak yazılmalıdır.

Program önce gelen komutun ne olduğunu anlar. Komuttan sonra gelen `A` ise dizinin adıdır, yani bir karakter dizisidir (string). Sonra ilgili fonksiyon çağrılır ve fonksiyon bağlı listeye ekleme, silme gibi işlemi yapar.

Soru: Bunları sanal olarak bellekte mi yapacağız, fiziksel olarak diskte değil, değil mi?

Cevap: Evet, tamamen sanal. Biri `mkdir A` dediğinde A diye bir dizin yaratılmış gibi yapılır. `dir` dendiğinde ekrana A yazılır. Disk üzerinde gerçekten bir şey yapılmaz.

Soru: A'nın altında bir şey oluşturmak için önce A'ya gidip sonra mı `mkdir` kullanmak gerekir?

Cevap: Evet, kesinlikle öyle. Bütün işlemler bulunulan dizinin bir altı veya bir üstü için yapılır.

11. Hata Durumları

Hoca, ödev metninde hangi hataların ele alınacağının yazılı olduğunu söyledi ve bunları tek tek geçti.

Komut	Hata durumu	Hocanın örneği
`mkdir`	Aynı seviyede aynı adlı dizin varsa	Kökte A, B, C varken `mkdir A`
`chdir`	Alt dizin yoksa	Kökte `chdir F`
`chdir ..`	Kökteyken, çünkü kökün üstü yok	A'dan köke çıktıktan sonra tekrar `chdir ..`
`rmdir`	O adda dizin yoksa veya altı doluysa	

Aynı ad farklı seviyelerde kullanılabilir. Kökte A varken B'nin altında da bir A açılabilir, bunda sorun yoktur. `dir` ve `count` yalnız bulunulan dizinin bir alt seviyesiyle ilgilidir: `dir` oradaki dizinleri gösterir, `count` onların sayısını verir. Hata mesajları da "o adda dizin yok" gibi basit mesajlardır.

Soru: `rmdir A` dediğimizde A'nın bütün çocukları ve kardeşleri de siliniyor mu?

Cevap: Hayır. Yalnız altı boş olan dizinler silinebilir.

12. Bir Üst Dizine Çıkmak

Soru: A'dayken köke çıkmak için `chdir ..` kullanıyoruz. Bunun için önceki yeri kaydeden ekstra bir işaretçi tutacak mıyız?

Cevap: Bunu kendiniz düşünmelisiniz. Tutmak isterseniz tutarsınız. Nerede bulunduğunuzu zaten tutuyorsunuz. Oraya nereden geldiğinizi de biliyor olmanız gerekir.

Soru: Yalnız ilk çocuğu tutuyoruz. C'nin altındayken de `chdir ..` dersek köke çıkması gerekir, değil mi?

Cevap: Evet. Kökte `chdir ..` hata verir, çünkü üstü yoktur. `chdir A` ile A'nın altına inip `chdir ..` derseniz köke çıkarsınız. `chdir B` ile B'nin altına inip `chdir ..` derseniz yine köke çıkarsınız. Bu adres elinizde olmalıdır, çünkü köke oradan geldiniz.

Hoca, her şeyin işaretçisini tutmak gerekmediğini söyledi. Bir üstü tutarak çözülebilir: `chdir` ile aşağı indiğinizde yalnız o an bulunduğunuz dizinin yerini bilmeniz yeter. Kökü de zaten tutarsınız, çünkü hiçbir şeyi kaybetmemek için kökü bilmek zorundasınız.

> **Ders sonrası not:** Hoca "bir üstü tutarak çözebilirsiniz" dedi. Tek bir üst adres değişkeni yalnız bir kez yukarı çıkmaya yeter. Hocanın örneğinde `A3_1` altına inildiğinde önce A3'e dönülür, ikinci `chdir ..` için ise A'nın adresi gerekir ve tek değişkende o adres artık yoktur; bu durum küçük bir C programıyla da görüldü. Birden fazla seviye iç içe ise üst adreslerin hepsi sırayla saklanmalı ya da üst dizin kökten başlanarak aranmalıdır.

13. Sınırlar: Tek Seviye Hareket ve Bellek

Soru: Yalnız bir üst dizine mi çıkıyoruz, ters bölü yazıp en üste çıkma var mı?

Cevap: Yok. `chdir /A/B` gibi bir komutla bulunulan yerden alakasız bir yere atlamak istenmiyor. Gerçek sistemde bu mümkündür, ama burada her şey bir üst ve bir alt seviyede çözülür.

Soru: Bir dizinin altında en fazla kaç dizin olabilir?

Cevap: Sınır yoktur. Yapı bağlı liste olduğu için tek sınır bellek ayırmadır (memory allocation). C'den sonra sağa doğru, sistem bellek ayırmaya izin vermeyene kadar ilerlenebilir. Bağlı liste zaten bu yüzden kullanılır.

14. Neden Aslında Bir Bağlı Liste Ödevi?

Hoca ödevin gözü korkutmaması gerektiğini birkaç kez vurguladı. "Bize işletim sistemi parçası yazdırdılar" diye düşünülebilir, ama yapılan iş ağaç yapısının bağlı listeyle gerçekleştirilmesidir. Komutların bağlı liste karşılığı şöyledir:

- `mkdir`: listeye yeni eleman eklemek (insert).
- `chdir`: liste üzerinde ilerlemek.
- `rmdir`: listeden eleman silmek.
- `dir`: listedeki elemanları ekranda göstermek.

Bağlı listede ekleme, silme ve bilgileri görme ilk haftalardan beri defalarca uygulandı. Burada yalnız hikâye farklı anlatılıyor. Amaç, bu işlemlerin gerçek bir uygulama alanında nasıl kullanılacağını görmektir. Hoca bunu şöyle bağladı: mezun olunca kimse size "bir diziyi büyükten küçüğe sıralayın" demeyecek, ama sıralama işlemini belki bir işin içinde kullanacaksınız.

Yapı biraz değişik göründüğü için tasarım üzerine de düşünmek gerekir. Şimdiye kadar bir sonraki veya bir önceki elemanın adresini gördük. Burada aynı yapının çocuk ve kardeş işaretçileriyle farklı bir kullanımı görülüyor.

> **Hocanın vurgusu:** Her katmanda kaç düğüm olacağını bilmiyorsanız bu yapıyı işaretçilerle bu şekilde göstermeniz gerekir. Ödevi mutlaka yapmaya çalışın, çoklu ağaçta çok faydasını görürsünüz.
>
> **Ders sonrası not:** Hoca bu durum için "başka çaremiz yok" dedi. Her düğümde çocuk adreslerini tutan, gerektikçe büyütülen bir dinamik dizi de kullanılabilir; bu nedenle first child / next sibling tek olası gösterim değildir, bu dersin kullandığı gösterimdir.

15. Algoritmik Ders Nasıl Çalışılır?

Hoca, veri yapılarında yeterince iyi olunmazsa üçüncü ve dördüncü sınıfta buna bağlı bütün derslerde zorlanılacağını söyledi. Bu nedenle çalışma yönteminin önemli olduğunu anlattı.

Algoritmik dersler konu okuyarak çalışılmaz, soru çözerek çalışılır. Derste bağlı listenin ekleme ve silme işlemlerini dinlemek yetmez, bunları kişinin kendisinin yazması gerekir. Sınavda çözülemeyen bir soru sonradan oturup yazılmalıdır: önce algoritması düşünülür, sonra derste

anlatılanlara bakmadan kodu yazılır. Kod yazarken hata yapılır, uğraşılır ve öğrenilen şey böylece kalıcı olur. Konular birbirinin üstüne eklendiği için bu yöntem sonraki derslerde de işe yarar.

> **Hocanın vurgusu:** Konu okuyarak değil, soru çözerek çalışın. Çözemediğiniz soruyu sonra kendi başınıza yazın. O zaman çok iyi öğrenirsiniz ve unutmazsınız.

Dersin Toparlaması

Quiz hataları, yapıların davranışını dikkatle izlemenin önemini gösterdi. Yığına itilen her eleman sonunda çekilir, bu yüzden pop sayısı push sayısına eşittir. Çift yönlü listede düğümün adresi silme için yeterlidir, çünkü önceki adres düğümün içindedir. Geri dönme gerektiren labirent problemi yığın mantığına oturur. Yığın, push ve ters çevirmeyle kuyruk gibi de kullanılabilir.

Dersin yeni kavramı first child / next sibling gösterimiydi. Her düğüm yalnız ilk çocuğunu ve sonraki kardeşini gösterir, böylece kaç çocuğu olacağı bilinmeyen bir ağaç bağlı listelerle kurulur. Ödevdeki sanal dizin sistemi bu gösterimi uygular. `mkdir`, `chdir`, `rmdir` ve `dir` komutları, aslında bağlı listede ekleme, ilerleme, silme ve listeleme işlemleridir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.